



* Always Remember $\rightarrow$ Try Your Best to come up with the Recursive logic only, Because after that DP is just 5 minutes of work. - vfliden

* Problem $\rightarrow$ height

Given an array `arr[]` of size `n`, where `arr[i]` denotes the height of i^{th} stone, given a value `k` which denotes the jump options you have from i^{th} index, for example:

Geek starts from stone 0 and from stone `i`, he can jump to stones `i + 1`, `i + 2`, ... `i + k`.

The cost for jumping from stone `i` to stone `j` is `abs(arr[i] - arr[j])`.

Find the **minimum** cost for Geek to reach the last stone.

$\rightarrow \text{abs}(\text{Height}[i] - \text{Height}[j])$

(stair) idx $\rightarrow$ 0 1 2 3

Input: `arr[] = [20, 30, 40, 20]`

Output: 20

$n=4, k=2$

Explanation: Minimum cost is incurred when the frog jumps from stair 0 to 1 then 1 to 3:

jump from stair 0 to 1: cost = $|30 - 20| = 10$

jump from stair 1 to 3: cost = $|20 - 30| = 10$

Total Cost = $10 + 10 = 20$

(index) stairs $\rightarrow$ 0 1 2 3
(values) Heights $\rightarrow$

20	30	40	20
----	----	----	----

 , $n=4$, $K=2$ This could vary

● $K=2$, we need to make two calls on each index

means you have 2 Jump options

(Remember LG - Greek Jump Problem)

$\text{int jump1step} = \text{abs}(\text{heights}[i] - \text{heights}[i+1]) + \text{Rec}(i+1, \text{heights})$

$\text{int jump2step} = \text{abs}(\text{heights}[i] - \text{heights}[i+2]) + \text{Rec}(i+2, \text{heights})$

